

From Policy Semantics to Safe Tool Execution: Action–Evidence Geometry and Execution Certificates

Anonymous Submission

Abstract

A well-formed tool call can still be unsupported by the trusted task, evidence, authorization, or current state. We present DAFR, an execution-layer compiler that represents each proposed call as an action–evidence point and evaluates it against state-dependent halfspaces and joint-risk regions. A deterministic typed field interface preserves the link from schema arguments to shared effect roles; the encoder records grounding, provenance, authorization, and risk coordinates; and the verifier emits a certificate with signed margins and violated relations. On the evaluated static attacks, DAFR obtains 0/585 observed attack successes on AgentDojo and 0/204 on Agent Security Benchmark, with 87.5% clean utility and 81.4% attack utility on AgentDojo and 86.8% utility on ASB. A matched 512-case study finds 25 false allows for independent axis thresholds, while an equivalent arithmetic predicate agrees with the geometric boundary on all cases. DAFR connects policy-defined joint risk to a common decision certificate, making the limiting execution relations explicit and available for action revision.

1 Introduction

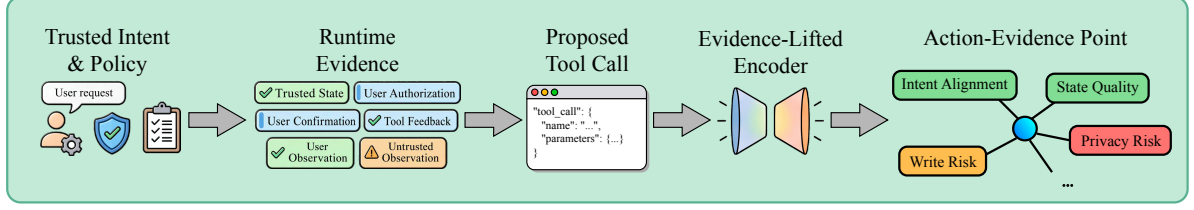
Tool-using agents can read private information, modify shared resources, contact external parties, and trigger actions that are difficult to reverse. These calls have more direct consequences than generated text: an executed call may immediately disclose information, change system state, or start an external operation (?????). Safe execution requires more than a valid schema. The function, effect-bearing arguments, expected effect, and state dependencies must be supported by the current task, available evidence, authorization, and policy.

Indirect prompt injection makes this decision difficult (???). An agent may need untrusted content to complete a valid task, yet that content can contain attacker-inserted instructions. The model can translate them into a well-formed, apparently relevant call that discloses sensitive information, modifies the wrong object, or creates an unauthorized external effect (?????). Tool descriptions and environment metadata provide additional channels for malicious control (?). The execution-time question is whether trusted evidence supports the call’s effect in the current state.

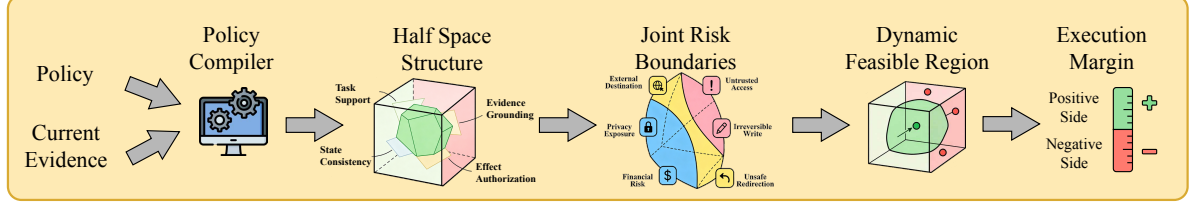
Consider an agent asked to update a customer record whose identifier is initially unknown. The agent retrieves the record, obtains its identifier from the verified result, and uses that identifier for the requested update. The retrieved content may also contain an attacker-issued command to send the record to a newly specified address. Both actions are schema-valid, but their support differs. The update follows the user request and uses an identifier established by the verified retrieval. The disclosure relies on a destination introduced through untrusted content and produces an unauthorized external effect. The execution layer must distinguish these actions by the evidence supporting their concrete effects.

Existing defenses act at different stages of the agent pipeline. Context-level methods separate trusted instructions from untrusted data, restructure prompts, repeat the user request, or filter suspicious observations before action generation (????). Runtime methods apply symbolic policies, privilege checks, execution plans, or information-flow rules to the generated action (????). Tool-action feasibility, however, often depends on relations among

Step 1: Build the Action-and-Evidence Point



Step 2: Compile the Dynamic Feasible Region



Step 3: Decide, Execute, and Update

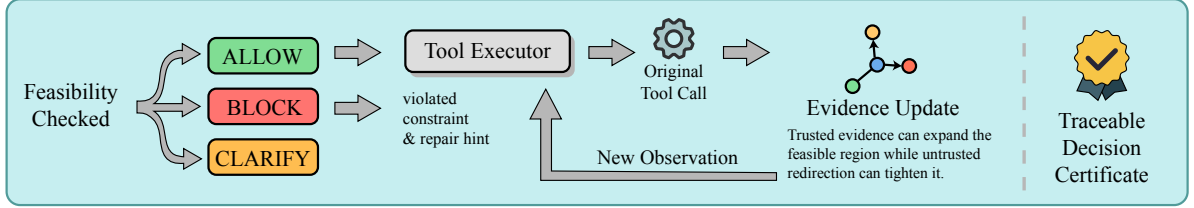


Figure 1: DAFR turns a proposed tool call, source-tagged evidence, and current state into an action–evidence point. Typed field validation preserves effect roles; the encoder records grounding, authorization, provenance, and risk; and the verifier evaluates active halfspaces and joint-risk regions. Region membership determines execution, while violated margins identify the relation used for clarification, repair, or blocking.

the action, its evidence, and the current state. A destination can be valid when specified by the user and invalid when copied from a web page. A write can become feasible after a verified read establishes its target. Combining sensitive data with an external destination can require stronger grounding and authorization than either factor alone.

We introduce DAFR (Dynamic Action Feasible Regions), an execution-layer compiler for deciding whether a concrete tool action is feasible at the current step. A deterministic typed field interface binds registered schema fields to shared effect roles and validates the policy envelope. An action–evidence encoder then grounds the call, runtime state, and source-tagged evidence in these roles. State-dependent halfspaces and joint-risk regions evaluate the resulting point and emit an execution certificate: a call inside the region proceeds, while a call outside it returns violated relations for clarification, repair, or blocking. This interface connects language-model proposals to explicit execution requirements and

preserves the evidence needed to inspect each decision.

The certificate changes with execution. A verified read can establish an identifier required by a later write, a trusted confirmation can authorize a scoped effect, and a new observation can invalidate an earlier state assumption. DAFR reconstructs the action–evidence point and active constraints before every external effect, retaining original field names and evidence links in the trace. The contribution is the composition of typed grounding, action–evidence encoding, joint-risk verification, and a certificate interface that exposes the limiting relation used for the next decision.

We evaluate DAFR on AgentDojo with three planner models and on the Agent Security Benchmark (ASB) with DeepSeek V4 Flash. DAFR obtains 0/585 and 0/204 observed attack successes on the two benchmarks, retaining 87.5% clean utility and 81.4% attack utility on AgentDojo and 86.8% utility and 73.5% task success on ASB. A system ablation shows that removing the com-

plete dynamic defense path increases observed ASR to 16.7%; a separate matched study isolates the narrower representation claim: independent axis checks miss coupled budgets, while an equivalent predicate agrees with the joint boundary and the geometric certificate exposes the limiting relations used for diagnosis and repair.

2 DAFR: Dynamic Action Feasible Regions

DAFR constructs an interpretable action–evidence point for the proposed call and instantiates the geometric constraints active in the current state. Region membership determines feasibility, and signed margins identify the relations that limit the decision.

2.1 Problem Setup

At step t , a tool-using agent receives a trusted user task u , operates under an execution policy p , observes the environment, and produces a concrete tool call where f_t is a registered tool and ρ_t maps the fields of the tool schema to argument values. The call is first checked against the schema. Passing this check establishes that the call is well formed and leaves its execution feasibility to the evidence and policy checks below.

Let E_t denote the execution state before a_t is evaluated. It contains the trusted task, execution policy, available tool schemas, established facts, completed prerequisites, authorizations, confirmations, and observations with source labels. Facts derived from external observations retain their source labels, while intent, confirmation, and authorization are recorded through trusted channels.

A call is feasible when the current evidence and policy support its function, the arguments that determine its external effect, the expected effect itself, and the required state conditions. We write when these conditions are satisfied. A feasible call is sent to the trusted runtime. An infeasible call is blocked or returned with feedback that identifies the function, argument, evidence source, authorization condition, or state assumption requiring revision. After an admitted call returns observation o_{t+1} , the state is updated as where T may add evidence, satisfy a prerequisite, record authorization, or invalidate an earlier assumption.

2.2 Typed Action–Evidence Compiler

The compiler separates typed field validation, action–evidence encoding, and execution verification. Let B_f bind the fields of registered tool f to a shared set of effect roles, let E encode a concrete call and its current evidence, and let G verify the resulting point. For a policy p and schema S_f , deterministic validation constructs where the interface records supported field roles, required prerequisites, authorization scope, and risk families. Unknown fields, unsupported effect types, inconsistent requirements, and out-of-range budgets are rejected before execution. Given a concrete call a_t and state E_t , the encoder computes retaining original field names and source links in the trace. The verifier returns where d_t is the execution decision and κ_t contains the active constraints, signed margins, provenance links, and revision targets. The three stages are therefore an auditable interface from a concrete language-model proposal to a deterministic execution decision.

2.3 Action–Evidence Point Construction

DAFR represents each proposed tool call and its current evidence as a point in an interpretable action–evidence space. The coordinates cover task support and progress, argument grounding and provenance, authorization and scope, and expected effects and risks.

DAFR uses the validated typed interface and action–evidence encoder to construct the action–evidence point: where L deterministically evaluates explicit execution relations. The runtime uses seven shared execution roles (object, destination, data, amount, time, effect, and scope) and a 45-dimensional action–evidence vector. Its compiler exposes 13 hard constraint templates and one soft confidence facet; coefficients and risk budgets are fixed globally for the evaluated configuration, while active role bindings and templates depend on the registered schema and current effect. A new tool therefore requires schema validation and an auditable field binding, not a new set of fitted geometric weights. The task block q_t records support for the selected function and its expected effect. The grounding block g_t connects effect-bearing arguments to source-

tagged evidence and established state. The authorization block c_t records confirmation, authority, and scope. The risk block r_t describes disclosure, transfer, deletion, redirection, and state change.

For an effectful call, the typed interface preserves the original field, value, grounding score, and source label. A destination introduced only by an untrusted observation therefore receives different provenance and grounding values from a destination specified by the trusted task, even when both calls satisfy the tool schema. This shared representation is the input to every comparison in the matched geometry study.

The evidence used by these relations is maintained in the execution state. After an admitted call returns o_{t+1} , an evidence projection operator converts it into source-tagged factual and risk records: where $g(v, E_t) \in [0, 1]$ measures the available support for value v . The minimum gives a conjunctive grounding summary across the arguments that determine the external effect: every critical value must satisfy its grounding requirement. Schema-optional fields follow the same effect-aware check. A field is eligible for omission during revision only when its removal leaves the destination, object, data, scope, authorization requirements, and expected external effect unchanged. Otherwise, an unsupported value keeps the call infeasible.

As evidence and execution state evolve, DAFR updates the affected coordinates, so the same proposed call may occupy a different position at a later execution step.

2.4 Dynamic Geometric Constraints

DAFR converts the execution requirements at step t into geometric regions in the action-evidence space. Each policy is compiled from the validated typed interface into frozen constraint templates. Individual requirements form halfspaces; coupled risks form a joint budget, for example $r_1^2 + r_2^2 \leq 1$. Independent if-else thresholds can accept (0.75, 0.75) when both thresholds are 0.8, even though the shared budget is exceeded. Tightening both thresholds to $1/\sqrt{2}$ would reject valid allocations such as (0.8, 0.2). The joint boundary represents this trade-off directly. An arithmetic predicate can implement the same

boundary, so the geometric contribution is the common certificate interface rather than greater program expressiveness.

For the active halfspaces and joint regions, DAFR computes signed margins and admits a call only when every active margin is non-negative. It records the violated conditions and normalizes their margins for feedback ordering. The limiting condition is mapped to a revision target such as an unsupported argument, unsuitable source, missing authorization, incomplete prerequisite, stale state assumption, or joint-risk excess. Each decision retains the concrete call, action-evidence point, active boundaries, margin vector, and evidence links, giving diagnosis and execution the same trace.

Each decision records the concrete call, its action-evidence point, the active boundaries, the margin vector, and the supporting evidence links. This record provides the same geometric basis for the execution decision and the revision target.

3 Evaluation

3.1 Experimental Setup

AgentDojo v1.2.2 is the primary benchmark and contains the Banking, Slack, Travel, and Workspace suites (?). We evaluate DeepSeek V4 Flash, GPT-5.4-mini, and Claude Haiku 4.5 on a fixed split of 71 clean and 585 attacked episodes: 16/144 in Banking, 21/105 in Slack, 20/140 in Travel, and 14/196 in Workspace. Clean and attack scores are unweighted macro-averages over suites. The supplement reports the complete baseline table and case counts. Component ablations evaluate evidence projection, action-evidence encoding, the dynamic defense path, and decision feedback. A blocked or malformed call remains in the benchmark denominator. For 0/585 and 0/204 observed attacks, Wilson 95% upper bounds are 0.65% and 1.85%, respectively.

We further evaluate DAFR on 204 ASB cases using DeepSeek V4 Flash (?). AgentDojo reports clean utility, attack utility, and attack success rate (ASR), while ASB reports utility, task success, and ASR.

DAFR is placed at the execution boundary, after the planner proposes a concrete

tool call and before the trusted runtime performs its external effect. The reported comparisons target matched case sets and planner models. The archived GPT baseline aggregates combine earlier three-suite document summaries with Workspace reruns; they are not uniformly fresh four-suite reruns. We disclose this provenance in the supplement and interpret differences descriptively. Under this setup, DAFR contributes the feasibility decision and relation-specific revision feedback for the proposed call, while the benchmark task and planning environment remain unchanged.

3.2 Main Results on AgentDojo

The AgentDojo comparison covers three planner models. With DeepSeek V4 Flash, DAFR achieves 87.5% clean utility and 81.4% attack utility with 0% observed ASR under the evaluated static attacks. DAFR also records 0% observed ASR under the evaluated static attacks with GPT-5.4-mini and Claude Haiku 4.5, while retaining 59.5% and 68.4% attack utility, respectively.

PIGuard is the only other representative method that reaches 0% observed ASR under the evaluated static attacks with all three planners. At this security level, DAFR improves attack utility over PIGuard by 38.9, 25.1, and 30.2 percentage points on DeepSeek, GPT, and Claude, respectively. These fixed-set comparisons show a favorable security–utility trade-off against PIGuard. They do not establish uniform dominance: with GPT, DAFR has lower attack utility than Prompt (65.4%) and Sandwich (65.4%), which have nonzero observed ASR; with Claude, Sandwich also has zero observed ASR and 67.1% attack utility, compared with 68.4% for DAFR.

3.3 Results on Agent Security Benchmark

Table 1 reports results on 204 ASB cases with DeepSeek V4 Flash. DAFR achieves 86.8% utility and 73.5% task success with 0% observed ASR under the evaluated static attacks.

Compared with no defense, DAFR reduces ASR from 90.2% to 0%, with a 1.9-point difference in utility. Among the evaluated methods, DAFR is the only one that reaches 0% observed ASR under the evaluated static attacks. Relative to Progent, the strongest competing

Method	Utility	Task Succ.	ASR
No Defense	88.7	77.9	90.2
Delimiter	87.3	77.0	86.3
Instruction Prevention	88.0	77.5	39.7
Observation Sandwich	88.0	77.0	27.9
TS-Flow	85.5	73.5	19.1
Progent	86.0	73.0	3.4
DRIFT	85.8	73.0	23.0
DAFR	86.8	73.5	0.0

Table 1: Results on 204 ASB cases with DeepSeek V4 Flash. Values are percentages; DAFR has 0/204 observed attack successes.

method by ASR, DAFR gains 0.8 points in utility and 0.5 points in task success and reduces ASR from 3.4% to 0%. The ASB results extend DAFR’s security–utility balance beyond AgentDojo to a distinct tool-use benchmark.

3.4 Ablation and Geometric Analysis

Table 2 reports the full AgentDojo system ablation with DeepSeek V4 Flash: each configuration uses the same 71 clean and 585 attacked cases across four suites, with suite-macro-averaged scores. These rows are system-level ablations because projection, encoding, geometry, and feedback interact. Removing evidence projection lowers clean utility from 87.5% to 79.2% while retaining 81.8% attack utility and 0% observed ASR under the evaluated static attacks. Removing action–evidence encoding raises clean and attack utility to 91.5% and 85.3% but introduces a 0.7% observed ASR.

Removing the complete dynamic defense path produces the largest security change in the original module ablation: observed ASR rises from 0% to 16.7%, with attack utility at 81.3%. This system ablation also changes observation handling and feedback, so it does not isolate the syntax of geometry. We therefore use a matched representation ablation: The membership controls share risk vectors and the repair controls share typed actions, evidence, and candidate transformations. The general predicate and geometric implementations are expected to agree on membership; the scientific comparison concerns axis-threshold expressiveness, margin-based diagnosis, and executable repair, not a claim that geometry can express functions unavailable to unrestricted

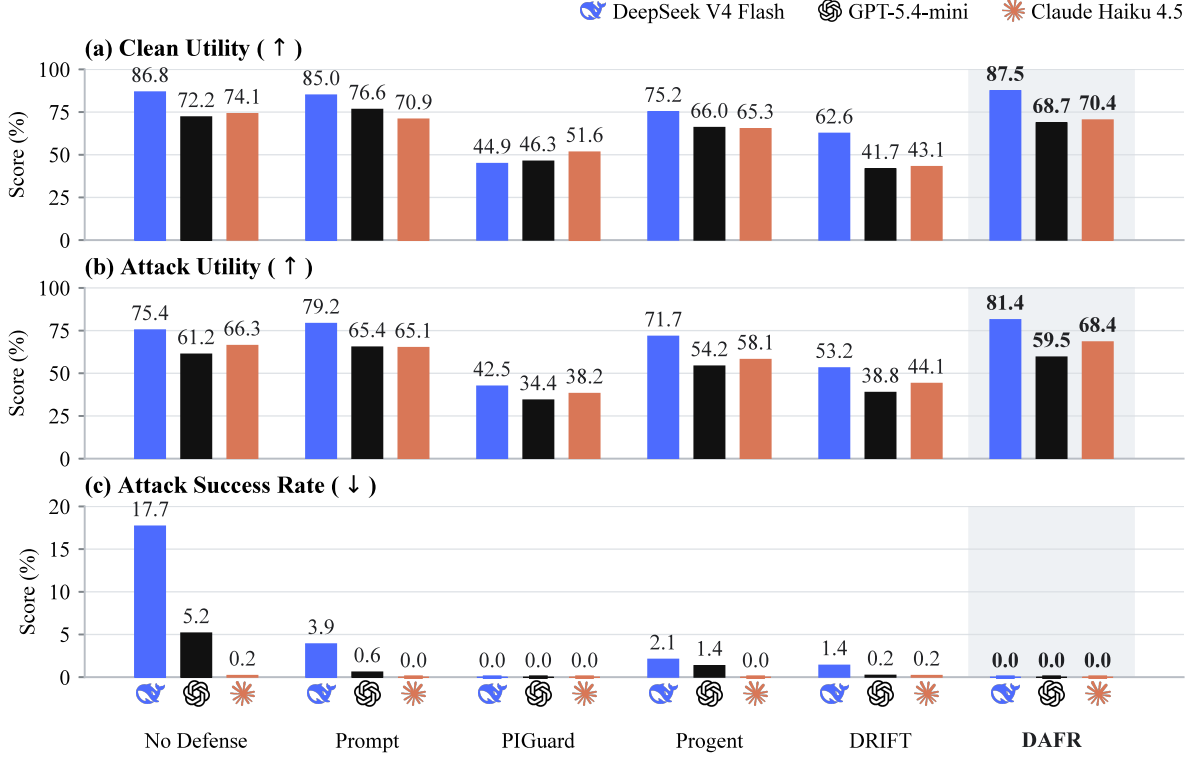


Figure 2: AgentDojo clean utility, attack utility, and attack success rate across representative methods and planner models. Values are percentages; the supplement reports the complete table and case counts.

programs. In a frozen 512-case joint-risk suite, independent axis thresholds allowed all cases and falsely allowed 25 actions whose joint risk exceeded the shared budget; the joint cone and equivalent predicate both allowed 487/512 and disagreed on 0/512. In a matched 256-case action suite, Geometry and Predicate+oracle each recovered 192 executable actions, and all 64 non-repairable actions remained blocked. A decision-only interface has no repair operator, so its zero recovery is a design property rather than an independently measured performance gap. This is evidence about coupled-budget encoding and an auditable repair interface, not arbitrary-program superiority. Removing decision feedback lowers clean and attack utility to 79.4% and 80.0% and introduces a 0.8% observed ASR.

Configuration	Clean	Attack	ASR
DAFR	87.5	81.4	0.0
w/o Evidence Projection	79.2	81.8	0.0
w/o Action-Evidence Encoding	91.5	85.3	0.7
w/o Dynamic Defense Path	90.2	81.3	16.7
w/o Decision & Feedback	79.4	80.0	0.8

Table 2: AgentDojo component ablation (percent).

Matched geometry protocol. We separate membership and repair controls. These matched controls are locally generated mechanism tests, rather than additional AgentDojo episodes: the 512 membership vectors are generated with fixed seed 20260930 in four dimensions, with every coordinate below the independent threshold 0.8 and analytic labels from the shared budget $lVertrVert_2 \leq 1$ (25 vectors violate this joint budget); the 256 repair cases are generated from a fixed local-write effect model. The membership study supplies identical risk vectors to independent axis thresholds, the joint-risk region, and an arithmetic predicate implementing the same joint boundary. The repair study supplies identical actions, evidence, and candidate transformations to the geometric and predicate backends. Predicate agreement is an implementation-equivalence check. The two studies test coupled-budget representation and action repair separately; neither isolates a geometric advantage over an equally instrumented predicate.

The matched 512-case joint-risk suite contains boundary-active four-dimensional risk

Backend	Safe allow	Unsafe block	False allow
Axis thresholds	487/487	0/25	25/25
Joint geometry	487/487	25/25	0/25
Equivalent predicate	487/487	25/25	0/25

Table 3: Membership under the specified unit-norm joint-risk policy. Labels are analytic; the matched suite is not an additional AgentDojo episode.

Backend + shared repair	Repaired	Required blocks
Geometry	192/192	64/64
Equivalent predicate	192/192	64/64

Table 4: Local action-repair audit. Both backends use the same candidate transformation and effect check.

vectors. Every coordinate remains below its independent threshold, but some vectors exceed the shared joint budget. The matched 256-case action suite supplies the same unsafe action and candidate repair to every arm; 192 cases are repairable by deleting an unsupported optional destination and 64 require blocking because the destination is semantically required. No case invokes an external tool or evolves from a completed benchmark trajectory. The matched study is therefore a representation and interface audit rather than a replacement for the end-to-end AgentDojo result. We also audited 614 saved AgentDojo decision records, of which 610 were well formed, by checking the recorded hard margins against an equivalent slack predicate. The two decisions agreed on 610/610 records; 546 were allowed and 64 were blocked or clarified. This is a certificate-consistency audit over saved boundaries, not a new planner replay or an end-to-end utility/ASR result. The matched study is reported separately because system ablations measure the complete defense path, whereas representation controls isolate coupled-risk handling and certificate observability.

4 Related Work

Prompt-injection defenses separate trusted instructions from untrusted observations, detect injected content, or compare normal and masked execution (?????). Runtime controls specify tool privileges, task-grounded trajectories, or information-flow labels (?????). DAFR operates at the execution boundary and binds concrete effect-bearing fields to

source-tagged evidence before a call is admitted.

Tool-use benchmarks study consequential failures and indirect injection across tool environments (?????). Action shielding restricts actions under formal safety specifications (?); DAFR contributes an auditable action-evidence certificate with state-dependent joint-risk margins and revision targets.

5 Conclusion

DAFR connects tool-call arguments to their supporting evidence and evaluates the resulting action-evidence point against state-dependent execution requirements. Its geometric representation composes joint-risk budgets with grounding and authorization constraints, while a shared certificate records the limiting relations and supports action revision. The retained benchmark results show a useful security-utility trade-off under static attacks. Matched controls establish the difference between independent thresholds and a joint budget, and confirm decision and repair equivalence with a predicate supplied with the same information and repair procedure. The contribution is a structured execution interface whose decisions and revision targets can be inspected together.

6 Limitations

DAFR depends on correct field bindings, explicit policy templates, and trusted provenance and authorization channels. Schema validity and region membership establish compliance with the encoded policy; neither proves that the policy or the encoder fully captures real-world safety. New effects may require manual bindings or templates. Fixed coefficients avoid online fitting but do not by themselves demonstrate robustness to parameter choice or eliminate development-set selection effects.

The benchmark evaluation uses fixed static attacks. The 64-case stress suite is a local, fixed-family test and does not establish resistance to adaptive, feedback-driven attackers. The 512-vector study uses analytically specified policy labels, and the 256-action repair suite varies a narrow optional-destination

transformation. The 610-record audit checks saved margins rather than newly executed trajectories. These studies establish controlled mechanism behavior, not generalization to arbitrary tools or repair tasks.

The retained multi-model tables have heterogeneous archival provenance, including document-derived baseline aggregates. Utility differences are descriptive and lack paired significance tests. The observed zero attack counts are limited to the evaluated cases; correlated attacks on shared tasks further limit population-level interpretation of binomial intervals. Full end-to-end latency, parameter sensitivity, and an independent adaptive-attack evaluation remain unmeasured in this manuscript.